

โครงการอบรมเชิงปฏิบัติการ Upskill/Reskill

เอกสารพัฒนาเฟิร์มแวร์ USB2ADC

ภาควิชาวิศวกรรมไฟฟ้าและคอมพิวเตอร์ คณะวิศวกรรมศาสตร์ มหาวิทยาลัยธรรมศาสตร์

2026-06-07

1. Firmware Requirement

USB2ADC คือเฟิร์มแวร์สำหรับบอร์ด NUCLEO-C562RE ที่ทำหน้าที่เป็นระบบบันทึกค่าแรงดันไฟฟ้าหลายช่องสัญญาณ (Multi-channel Voltage Logger) โดยออกแบบการสื่อสารตามมาตรฐาน SCPI เพื่อให้รองรับการทำงานร่วมกับซอฟต์แวร์มาตรฐานอุตสาหกรรม เช่น LabVIEW (ผ่าน NI-VISA), MATLAB หรือ Python (PyVISA)

1.1. รายละเอียดฮาร์ดแวร์

การเชื่อมต่อฮาร์ดแวร์บนบอร์ด NUCLEO-C562RE:

ขาบนบอร์ด	ฟังก์ชัน	รายละเอียด
A0 - A3	ADC Input	ช่องรับสัญญาณแอนะล็อก (0 - 3.3V) A0: ขา PA0, A1: ขา PA1, A2: ขา PA4, A3: ขา PB0
USB Connector	USB-Virtual COM Port	การสื่อสาร UART2 (115200, 8, N, 1) ผ่าน ST-Link
PA5	User LED	ไฟแสดงสถานะการทำงาน

1.2 ภาพรวมของ SCPI Command

คำสั่งพื้นฐานสำหรับการตั้งค่าและอ่านค่าจากอุปกรณ์ USB2ADC สรุปได้ตามตารางด้านล่าง:

คำสั่ง	คำอธิบาย	พารามิเตอร์	ค่าเริ่มต้น
*IDN?	สอบถามชื่อและเวอร์ชันของอุปกรณ์	-	-
*RST	รีเซ็ตการตั้งค่าเฟิร์มแวร์เป็นค่าเริ่มต้น	-	-
:MEASure:VOLTage?	อ่านค่าแรงดันไฟฟ้าทุกช่อง (A0-A3) แบบครั้งเดียว	-	-
:MEASure:VOLTage:CH? x	อ่านค่าแรงดันเฉพาะช่องที่ระบุ	0 1 2 3	0
:CONFigure:RATE x	ตั้งค่าความถี่ในการ Sampling (Hz)	1 ถึง 100	10
:CONFigure:RATE?	สอบถามความถี่การ Sampling ปัจจุบัน	-	-

คำสั่ง	คำอธิบาย	พารามิเตอร์	ค่าเริ่มต้น
:FORMat:DATA x	เลือกรูปแบบสตริงที่ใช้ส่งข้อมูล	ASCIi JSON	ASCIi
:INITiate:CONTinuous	เปิด/ปิด โหมดส่งข้อมูลต่อเนื่อง (Streaming)	ON OFF 1 0	OFF
:ABORt	หยุดการส่งข้อมูล Streaming ทันที	-	-

1.3 ข้อกำหนดทางเทคนิค

- **Sampling Rate:** สูงสุด 100 Hz (คาบเวลาน้อยสุด 10ms)
- **Resolution:** 12-bit (0 - 4095) แปลงเป็นทศนิยม 0.00 - 3.30V
- **Termination:** Line Feed (\n) หรือ Carriage Return + Line Feed (\r\n)

1.4 Functional Constraints

1. **Sampling Determinism:** ระบบต้องใช้ Hardware Timer ในการ Trigger ADC เพื่อรักษา Jitter ให้ต่ำที่สุดตามความถี่ที่ตั้งไว้
2. **Buffer Management:** การรับคำสั่ง SCPI ต้องใช้ Interrupt-driven UART ร่วมกับ Input Buffer ขนาดไม่น้อยกว่า 64 Bytes เพื่อป้องกันคำสั่งตกหล่น
3. **Voltage values:** ค่าแรงดันเป็น text เลขทศนิยม 2 ตำแหน่ง 0.00 - 3.30
4. **Response Time:** อุปกรณ์ต้องตอบกลับคำสั่ง Query (ที่มีเครื่องหมาย ?) ภายในเวลาไม่เกิน 10ms

2. Firmware Design

การออกแบบเฟิร์มแวร์ USB2ADC เน้น โครงสร้างที่แยกส่วนหน้าที่ (Separation of Concerns) เพื่อให้ง่ายต่อการแก้ไขพารามิเตอร์ และการจัดการคำสั่ง SCPI ที่มีความซับซ้อนโดยไม่กระทบต่อความเที่ยงตรงของจังหวะการสุ่มตัวอย่างข้อมูล (Sampling Determinism)

2.1 Software Architecture

สถาปัตยกรรมซอฟต์แวร์แบ่งออกเป็น 3 เลเยอร์หลัก เพื่อแยกฮาร์ดแวร์ออกจากลอจิกของแอปพลิเคชัน:

1. **Application Layer:** ประกอบด้วย SCPI_Cmd_Parser(), SCPI_Execute() และ SCPI_Error() ทำหน้าที่ตีความคำสั่งจากผู้ใช้และตัดสินใจลำดับการทำงาน รวมถึงการรายงานข้อความ error
2. **Service Layer (Middleware):** จัดการ ระบบ Buffer, การจัดการ หน่วยความจำ สำหรับ ADC Data และฟังก์ชันการแปลงค่า (Scaling) จาก Raw Data เป็น Voltage (V)
3. **Hardware Abstraction Layer (HAL):** ใช้ STM32Cube HAL ในการควบคุม Peripheral (ADC, TIM, UART, DMA) โดยเลเยอร์นี้จะติดต่อกับรีจิสเตอร์ของ MCU โดยตรง

2.2 Behavior Design: Main State Machine

การทำงานหลักของ USB2ADC ถูกควบคุมด้วย State Machine ที่รองรับทั้งการวัดค่าแบบครั้งเดียว (On-demand) และการส่งข้อมูลต่อเนื่อง (Streaming) เพื่อให้การจัดการทรัพยากรระบบเป็นไปอย่างมีประสิทธิภาพ:

เหตุการณ์ / สถานะ	BOOT	IDLE	MEASURE	STREAM	ERROR
auto	IDLE / init HW	-	-	-	-

เหตุการณ์ / สถานะ	BOOT	IDLE	MEASURE	STREAM	ERROR
:MEAS	—	MEASURE / start ADC	—	—	—
auto	—	—	IDLE / report data	—	—
:CONT:ON	—	STREAM / init TIM > ADC	—	—	—
:CONT:OFF หรือ :ABORT	—	—	—	IDLE / reset TIM > ADC	—
:FORMAT	—	IDLE / update config	—	IDLE / reset TIM > ADC & update config	—
error	—	ERROR / blink LED	ERROR / blink LED	ERROR / blink LED	—
RST	—	BOOT / SW reset	BOOT / reset	BOOT / reset	BOOT / reset

2.3 SCPI Command: Parser State Machine

การรับคำสั่งแบบข้อความ (Text-based) ถูกออกแบบให้ทำงานแบบ Character-by-Character ภายใน Interrupt Service Routine (ISR) โดยใช้ **state machine** แยกการเก็บ Command และ Parameter ออกจากกัน และใช้ **\n (Line Feed)** เป็นตัวจบคำสั่งเท่านั้น (อักขระ \r จะถูกละเว้น)

ขั้นตอนการทำงานของ Parser:

1. **WAIT_HEADER:** รอรับตัวอักษรแรกของคำสั่ง ซึ่งต้องเป็น : (สำหรับคำสั่งเฉพาะระบบ) หรือ * (สำหรับคำสั่งสากล) เท่านั้น หากได้รับอักขระอื่น ระบบจะล้าง Buffer และรีเซ็ตสถานะทั้งหมด
2. **COLLECT_MNEMONIC:** เริ่มเก็บอักขระของคำสั่งลงใน cmd buffer ทีละตัว จนกว่าจะพบ:
 - ช่องว่าง (): เปลี่ยนไปสถานะรอพารามิเตอร์
 - เครื่องหมายคำถาม (?): เก็บ ? ลง cmd buffer พร้อมตั้ง is_query = true แล้วเปลี่ยนไปสถานะรอพารามิเตอร์
 - ตัวจบคำสั่ง (\n): สิ้นสุดการรับ
3. **COLLECT_PARAMETER:** เมื่อเจอช่องว่าง () ระบบจะเริ่มเก็บอักขระที่เหลือ (รวมถึงตัวเลข, ตัวอักษร, ช่องว่างเพิ่มเติม) ลงใน param buffer โดยไม่มีการตีความ จนกว่าจะพบตัวจบคำสั่ง (\n)
4. **DISPATCH:** เมื่อพบ \n ระบบจะ
 - ใส่ null-terminator ที่ท้าย cmd buffer และ param buffer
 - รีเซ็ต state machine กลับสู่ WAIT_HEADER (เพื่อรับคำสั่งถัดไป)

หมายเหตุ:

- ระบบรองรับทั้ง \n (LF) และ \r\n (CR+LF) โดย \r จะถูก ละเว้น (ไม่เก็บ ไม่ทำให้จบ) ทำให้ parser ทำงานได้กับ terminal ที่ตั้งค่า line ending แบบ LF หรือ CRLF
- หากได้รับอักขระเกินขนาด buffer หรือมีคำสั่งใหม่เข้ามาขณะที่คำสั่งเดิมยังไม่เสร็จ ระบบจะคืนสถานะ SCPI_STATUS_ERROR

2.4 Timing Analysis

2.4.1 UART command/data transfer

การวิเคราะห์เงื่อนไขทาง timing ของระบบ USB2ADC เมื่อกำหนด UART2 ที่ 115200 bps เพื่อส่งข้อความ CSV (ยาวสุด 21 ไบต์ 1.82 ms) หรือ JSON (ยาวสุด 45 ไบต์ 3.90 ms) มีขีดความสามารถในการทำงานดังนี้:

รายการ	ค่าที่ออกแบบ	ผลการประเมิน	หมายเหตุ
Max Throughput	11,520 Bytes/s	ผ่าน	เพียงพอต่อการส่ง JSON (max 4.5 KB ที่ 100Hz)
Baud rate Stability	115,200 bps	ผ่าน	แนะนำให้ใช้ DMA ในการส่งเพื่อลดภาระ CPU ใน Main Loop
Loop Determinism	10ms Cycle	เตือน	ห้ามใช้ HAL_Delay(10) แบบ Blocking ให้ใช้ Timer Flag แทน
Precision	Float 2 digits	ผ่าน	ใช้พื้นที่ใน Buffer เหมาะสม (4 Bytes ต่อค่าแรงดัน)

คำแนะนำเพิ่มเติม: ในโหมด JSON หากในอนาคตมีการเพิ่มจำนวนช่อง ADC (เช่น A0-A7) จะต้องปรับเพิ่ม Baud rate เป็น 230,400 หรือลดความถี่ในการส่งลง เพื่อป้องกันไม่ให้ความยาวของสตริงเกินขีดจำกัดของเวลาในหนึ่งรอบการทำงาน

2.4.2 ADC data flow and synchronization

การแปลงค่า ADC 4 ช่องจะทำให้เกิด data overrun หากไม่ย้ายข้อมูลออกจาก ADC_DR เมื่อเกิด EOC (end-of-conversion) จึงใช้ DMA ในการย้ายข้อมูลจาก ADC ไปยัง Buffer เพื่อลดภาระของ CPU

- **Timer to ADC:** ใช้ Hardware Trigger จาก Timer เพื่อควบคุมอัตราการสุ่มตัวอย่าง (สูงสุด 100 Hz) ให้คงที่
- **Safe Data Access:** ใช้ Double Buffering หรือ Flag Management เพื่อป้องกันปัญหา Race Condition ระหว่างเลเซอร์ที่ประมวลผลข้อมูลและเลเซอร์ที่ส่งข้อมูลออก

Feature	Configuration	Concepts
Number of Conversion	4	กำหนดจำนวนช่องสัญญาณ (A0 - A3)
Scan Conversion Mode	Enabled	เพื่อให้ ADC วนอ่านทุกช่องในหนึ่งรอบ
Continuous Conv Mode	Disabled	ให้ Timer (TIM) เป็นตัวคุมจังหวะแทน
DMA Continuous Requests	Enabled	ให้ DMA พร้อมรับข้อมูลใหม่เสมอเมื่อมีการสแกน
External Trigger Source	Timer 3 Out Trigger	เพื่อให้จังหวะการวัดคงที่ (Deterministic)

3. Firmware Detailed Design

สถาปัตยกรรมของเฟิร์มแวร์ถูกออกแบบตามมาตรฐาน Layered Architecture โดยใช้ STM32Cube HAL (Hardware Abstraction Layer) เป็นฐาน และแยกส่วนตรรกะ ออกจากโค้ดที่สร้างโดย STM32CubeMX2 เพื่อให้ง่ายต่อการบำรุงรักษา

3.1 Firmware Architecture

การจัดโครงสร้างไฟล์ (Modular Programming) เพื่อแยกหน้าที่แต่ละส่วนออกจากกัน (Separation of Concerns):

Files	Responsibility	Key Functions
main.c	ควบคุม Main Loop และการเปลี่ยนผ่านสถานะ (State Transition)	main(), Error_Handler()
scpi.c/h	ตีความคำสั่งจากพอร์ตซีเรียลและจัดการ Command Event	SCPI_Parse_Char(), SCPI_Execute(), SCPI_Error()
adc_service.c/h	ควบคุมการวัดค่า ADC, การแปลงหน่วย (Scaling) และการจัดรูปแบบข้อมูล	ADC_Start_Scan(), Format_Payload()
app_config.h	รวบรวมค่ากำหนดคงที่ (Constants) และ Macro ต่างๆ	#define MAX_BUF_SIZE 64
stm32c5xx_it.c	จัดการงานที่เกิดขึ้นใน Interrupt (Background Task)	USART2_IRQHandler(), TIM3_IRQHandler()

3.2 Hardware Configuration

การตั้งค่า Peripherals บน STM32CubeMX2 สำหรับโปรเจกต์ USB2ADC:

Peripheral	Mode	Key Settings	Purpose
ADC1	Regular Scan Mode	A0-A3 (4 Channels), Rank 1-4	อ่านค่าแอนะล็อก 4 ช่องพร้อมกัน
DMA1	ADC to Memory	Circular Mode, Increment Memory	โอนข้อมูลเข้า RAM โดยไม่ใช้ CPU
TIM3	Output Trigger	Period=10ms, TRGO on Update	เป็นฐานเวลาคงที่ (100Hz) ให้ ADC
USART2	Asynchronous	115200 8N1, TX/RX Interrupt	สื่อสาร SCPI ผ่าน Virtual COM Port
SYS/GPIO	Output	PA5 (User LED)	แสดงสถานะเครื่อง (Heartbeat) และ Error

3.3 Main Loop Execution

กระบวนการประมวลผลภายในลูปลหลักจะทำงานแบบ **Cyclic Executive** ที่ความถี่ 100 Hz (10ms):

- Cycle Start:** รอสัญญาณ Flag จาก Timer (10ms) เพื่อเริ่มรอบการทำงาน
- Command Polling:** ตรวจสอบ SCPI_Event_ready_flag ว่ามีคำสั่งใหม่เข้ามาหรือไม่
- State Logic Update:** หากมีคำสั่งใหม่ ให้ประมวลผลเพื่อเปลี่ยน System_State (เช่น จาก IDLE เป็น STREAM)
- Data Acquisition/Formatting/Report:** อ่านข้อมูลจาก DMA Buffer, แปลงเป็นแรงดัน (Volt) และจัดรูปแบบเป็น CSV หรือ JSON, ส่งผลลัพธ์ออกทาง UART
- End of Cycle:** ล้าง Flag และเตรียมตัวสำหรับรอบถัดไป

3.4 Implementation Details

3.4.1 Main State Machine

การระบุสถานะและตัวแปรควบคุมในระดับซอฟต์แวร์:

```

/* Enumeration of System States */
typedef enum {
    SCPI_STATE_BOOT = 0,
    SCPI_STATE_IDLE,
    SCPI_STATE_MEASURE,
    SCPI_STATE_STREAM,
    SCPI_STATE_ERROR
} scpi_state_t;

/* Global State Variable */
scpi_state_t current_state = STATE_BOOT;

```

3.4.2 SCPI Command Parser Flow

โครงสร้างข้อมูลสำหรับการจัดการเหตุการณ์คำสั่ง SCPI:

```

typedef struct {
    char cmd_buf[MAX_SCPI_BUF_SIZE];    // command string
    char param_buf[MAX_SCPI_BUF_SIZE];  // parameter string
    uint8_t cmd_buf_index;              // cmd_buf index
    uint8_t param_buf_index;            // param_buf index
    bool ready_flag;                    // Terminator (\n) found
    bool is_query;                      // query command (?)
} scpi_cmd_t;

```

4. Firmware Testing

การทดสอบเฟิร์มแวร์ USB2ADC มุ่งเน้นไปที่การตรวจสอบความเที่ยงตรงของจังหวะการวัด (Timing) ความถูกต้องของโปรโตคอล SCPI และความเสถียรในการส่งข้อมูลต่อเนื่อง (Streaming)

4.1 Test Environment

การเตรียมสภาพแวดล้อมสำหรับการทดสอบ (Test Setup) เพื่อจำลองการใช้งานจริงและตรวจสอบความถูกต้องของข้อมูล:

1. Hardware Setup:

- **Signal Source:** ใช้แหล่งจ่ายแรงดันอ้างอิง (Variable DC Power Supply) หรือวงจร Voltage Divider เพื่อป้อนแรงดัน 0-3.3V เข้าที่ขา A0-A3
- **Observation Tools:** ใช้ Logic Analyzer หรือ Oscilloscope ในการวัดสัญญาณที่ขา PA5 (Heartbeat LED) เพื่อตรวจสอบ Loop Time (10ms)

2. Software Setup:

- **Serial Terminal:** ใช้โปรแกรม เช่น PuTTY, Tera Term หรือ RealTerm (ตั้งค่า Baud rate 115200)
- **Test Scripts:** เตรียม Python Script (ใช้ PySerial) สำหรับส่งคำสั่ง SCPI แบบอัตโนมัติ
- **Monitoring:** ใช้ซอฟต์แวร์ LabVIEW หรือ Python (Matplotlib) ในการพลอตค่าแรงดันที่ได้รับจากโหมด Streaming

4.2 Unit Testing

ตารางสรุปการทดสอบระดับหน่วย (Unit Test) เพื่อยืนยันฟังก์ชันการทำงานพื้นฐานของเฟิร์มแวร์:

ID	Objective	Pre-req	Steps	Input	Expected Output
UT-01	ตรวจสอบการตอบสนองพื้นฐาน	เชื่อมต่อ USB VCP	ส่งคำสั่ง IDN ผ่าน Terminal	*IDN?	TU,USB2ADC,V1.0\n
UT-02	ตรวจสอบความแม่นยำ ADC	ป้อนแรงดัน 1.65V ที่ A0	ส่งคำสั่งวัดค่าช่อง 0	:MEAS:VOLT:CH?1.65 (±0.05V) 0	
UT-03	ตรวจสอบการเปลี่ยน Format	สถานะ IDLE	เปลี่ยนเป็น JSON และวัดค่า	:FORM:DATA JSON แล้วส่ง :MEAS?	{"A0":x.xx, ...}
UT-04	ตรวจสอบขอบเขตการตั้งค่า	สถานะ IDLE	ตั้งค่า Rate เกินขีดจำกัด	:CONF:RATE 2000	ERR: -222, "Data out of range"
UT-05	ตรวจสอบระบบ Reset	เปลี่ยน Config เดิม	ส่งคำสั่ง Reset ระบบ	*RST	การตั้งค่ากลับเป็น Default (ASCII, 100Hz)

4.3 Integration Testing

การทดสอบความสามารถในการทำงานร่วมกันระหว่าง โมดูล (Module Interoperability) และ การทำงานภายใต้เงื่อนไขเวลา:

4.3.1 SCPI-Stream Integration

- **Purpose:** ตรวจสอบว่าระบบสามารถรับคำสั่งจัดจังหวะในขณะที่กำลัง Streaming ได้หรือไม่
- **Procedure:**
 1. ตั้งให้ระบบทำงานในโหมด Streaming (:INIT:CONT ON)
 2. ในขณะที่ข้อมูลกำลังไหล ให้ส่งคำสั่ง ABORT แทรกเข้าไป
- **Expected Result:** ข้อมูล Streaming ต้องหยุดทันที และระบบต้องกลับสู่สถานะ IDLE พร้อมรับคำสั่งใหม่

4.3.2 Timing & Throughput Stability

- **Purpose:** ตรวจสอบอาการ Jitter ของข้อมูลเมื่อใช้ JSON Format ที่ 100 Hz
- **Procedure:**
 1. ตั้งค่าอัตราเร็วสูงสุด 100 Hz และรูปแบบข้อมูล JSON
 2. ทำการ Streaming ต่อเนื่องเป็นเวลา 10 นาที
 3. ตรวจสอบจำนวนบรรทัดของข้อมูลที่ได้รับ (ต้องครบ 60,000 ชุด)
- **Expected Result:** ข้อมูลไม่ขาดหาย และไม่มีตัวอักษรผิดเพี้ยน (No Buffer Overflow)

4.3.3 Host Application Connectivity (LabVIEW/Python)

- **Purpose:** ตรวจสอบการเชื่อมต่อกับ Standard VISA Library
- **Procedure:** ใช้ LabVIEW VISA Read/Write ในการรับส่งข้อมูลตาม Command Set
- **Expected Result:** แอปพลิเคชันสามารถ Parse ข้อมูลได้ถูกต้องตาม Format ที่ระบุ และสามารถควบคุมอุปกรณ์ได้สมบูรณ์